BUILDING WITH IBM BOB · THEME 1: EXPLORE, FIX, AND BUILD

OnRamp

An IBM Bob kit that takes a developer from "never seen this repo" to a verified fix and feature, with evidence behind every claim.

Team

Solo participant
Code: github.com/KnigguKniggu-droid/onramp
Evidence: knigguKniggu-droid.github.io/onramp-evidence

Built with

IBM Bob IDE 2.2.1 (Plan, Agent, 3 custom modes, skills, commands, hook) and Bob Shell 2.0.5 · Python · Django · Flask · Node.js · GitHub Actions

THE DEVELOPER PROBLEM

In a real codebase, red tests don't tell you whether you broke something.

- Newcomers inherit large repositories, local quirks and test suites that already fail on their machine.
- Healthchecks (10k-star Django app): **29 of 1,761 tests fail on Windows** before anyone changes a line.
- AI assistants make plausible changes fast, but "it passes" is not evidence when the baseline is red.

Question: can IBM Bob onboard onto unfamiliar code, fix a real bug and add a real feature, so that every claim can be checked?

Why it matters

Brownfield onboarding is where developers lose the most time, and where trust in an AI partner is earned or lost. OnRamp makes Bob's work reproducible: baseline first, failing test before fix, and a report a reviewer can verify.

WHAT WE BUILT

OnRamp: one folder you drop into a repository

/onramp

Validates the install, writes a repo profile, captures a baseline, maps the codebase.

Explorer mode

Read-only guide with file references; cannot edit or run anything.

Reproducer mode

Failing test first; the fix comes only after the pre-fix artifact exists.

Verifier mode

Re-runs tests and classifies every failure: inherited, regression, fixed, unchanged.

Evidence

Stdlib generator: sealed blind benchmarks, attached reference patches, REPORT_<slug>.md.

Built entirely from Bob extension points: `.bob/custom_modes.yaml`, 4 skills, 2 commands, a PostToolUse hook in `.bob/settings.json` and `.bob/gen_report.py` (65 tests). After Bob's allowance ran out, Claude Code added a separately labelled normalizer/bootstrap module (`onramp_normalize.py`, 20 tests).

RESULT 1 · EXPLORE

29 failures, zero regressions

1,761
tests in the Healthchecks baseline

29
inherited failures, recorded before any change

1,766
tests after Bob's fix and feature

0
regressions, same 29 failing test IDs

- 26 Signal tests need `socket.AF_UNIX`, 2 use a Linux-only date format, 1 DST test needs time-zone switching. All are Windows-only, and all are classified as inherited.
- Bob generated `CODEBASE_MAP.md` with its map-codebase skill, and Claude Code checked the baseline against an independent run (identical IDs).

RESULT 2 · FIX (BLIND)

Bob vs the maintainer

- Healthchecks pinned at upstream `0968fa2` (the benchmark itself was sealed at kit commit `a3acbc3`), the commit before the maintainer fixed a real bug: webhooks answering HTTP 205 were retried 3 times and marked failed.
- Bob got only the symptoms. The brief was SHA-256 sealed before the issue-specific work began (Bob had already mapped the codebase).
- Reproducer: the failing test (`3 != 1` requests) came first, then a 1-line fix. Independent full-suite run: 0 regressions.
- Only after Bob committed did we reveal upstream `0a9bff9`.

Honest comparison (written by Bob)

Same defect, same line, different policy. Our brief said "a 2xx response such as 205" should succeed; Bob accepts every 2xx response; the maintainer added only 205. Bob's own report concludes the narrower upstream fix is safer, because `HttpTransport` is shared by about 25 integrations.

RESULT 3 · BUILD AND REUSE

A real feature, and a kit that is not hardcoded

Feature #1090 (open since 2024)

JSON webhook bodies are shown pretty-printed in the ping details view; non-JSON and email bodies are untouched, and Download Original still serves the exact bytes. An independent check found `json.dumps` escaping "café"; Bob added a test and fixed it.

Portability on a Flask app (TaskFlow)

The kit was copied unchanged three times. Run 1 exposed Django assumptions (hardcoded paths, output parsing, file names). Bob made the kit profile-driven. Run 2 then found one parser gap, and after Bob's fix run 3 classified the failure correctly.

> Every gap went back to Bob as an observed failure while Bob had budget; a guard mod in Claude Code blocked Claude from editing Bob's files. After Bob's 180-Bobcoin allowance ran out, remaining fixes were added as separate, Claude-labelled files.

HOW BOB WAS USED

IBM Bob as the engineering partner, end to end

- **Plan mode** designed the kit and asked 4 design questions before building.
- **Agent mode** built the modes, skills, commands, hook and generator, each in its own commit.
- **Custom modes** (Reproducer, Verifier) drove the blind fix.
- **Bob Shell (ACP)** ran the portability tests in other repositories.

Process, honestly

Claude Code operated Bob at the participant's direction, verified each result independently (full suites, revert checks, baseline diffs) and reported only observed failures back. Every one of Bob's commits ends with `Generated-by: IBM Bob`.

ROADMAP AND LEARNINGS

What we learned, and what comes next

Learnings

- A baseline is the difference between "tests are red" and "I broke something".
- Sealing the brief makes an AI fix comparable with a human one.
- Bob's first plan guessed config locations; testing the kit on a second repository exposed what a single repo hides.
- Budget discipline: small, fresh Bob tasks cost a fraction of long ones.

Roadmap

- Profiles for pytest, Node and Java projects.
- Record allowed and withheld information in every sealed benchmark.
- A CI job that runs `gen_report.py classify` on every pull request.
- More blind benchmarks from real upstream fixes, to score agents over time.